

總覽：agent 拆成模塊，各一條只講性質的規格；沒拆就搬不走、驗不了哪裡錯

這份文件回答：agent 的能力能不能拆成可復用的基本模塊；模塊怎麼描述；有和沒有模塊化差在哪

模塊 = agent (自主運行的 AI 程式) 的一項能力，和 design 的 module 無關；每個模塊一條規格。

規格 (軟體說的 contract) 只講它建立什麼性質、回答什麼問題；不講讀哪份文件、怎麼跑測試。

例：乾淨環境重現——主要文件是否描述了怎麼重現執行，而且結果有 fail-safe 的辨認方式 (過 / 不過 / 探測本身壞了)。

沒有模塊化：agent 是一整塊

CI/CD mentor agent

接 depot 的程式 · 監看 · 分析 · 溝通 · shelved CL · check · 日誌 · 請示...
功能和機制綁在一起：讀哪份文件、怎麼跑測試都寫死在裡面

- x 目標 depot 換成 git，要重寫一半
- x 別的 agent 一塊都拿不走
- x 沙盒只能驗整體：過了不知道哪裡對，錯了不知道哪裡錯
- x 改一處牽全身；行為講不清楚，也驗不了

有模塊化：三層

agent = 模塊的組合；公司專屬的只有最下層

CI/CD mentor agent

資安 agent

coding 規範 agent

將來別的 agent：同樣的模塊，換評分表

十七個模塊

看 4 判 3 做 3 說 3 守 4

看判做說守是 agent 的五種動作；每個模塊一條規格 + 沙盒裡一組驗它的測試

接 depot 的程式

Perforce / git

評分表

六原則、資安、規範...

規矩表

授權、登記、症狀目錄

公司專屬的只有這三樣：換公司、換版控只換這一層

- ✓ 規格可測：沙盒 (埋了已知問題的假 depot) 逐模塊驗
- ✓ 別的 agent 換評分表、規矩表就能用；換 git 只換接 depot 的程式

agent 的能力拆成十七個模塊，每個一條只講性質的規格，所以測得了、搬得走；公司專屬的只有三樣；沙盒驗的是規格。

後面：五頁列模塊 (看、判、做、說、守)，一頁接公司的三樣與復用，一頁 mentor agent 的十個階段各用哪些模塊。

看：agent 用四個模塊認識 depot，每條觀察附出處，猜的不能當依據

模塊 用在哪	規格（建立什麼性質；不講機制）	進出 沙盒怎麼驗
結構地圖 PROJECT_MAP 盤點的熱點	只讀 depot 就說得出每個目錄的用途、誰引用誰、哪些沒人引用 每一筆標依據；沒人引用的只標不刪	目錄樹+引用 (filelist、include) → 地圖、未引用清單 沙盒：埋一個沒人引用的目錄，地圖要標出來且不刪
變更流觀察 盤點、日常監看 提醒級 check	從變更歷史推出行為訊號：包多大、說明有沒有目的、多久進一次、 誰動哪裡、有沒有用 branch；對照症狀目錄發事件 症狀目錄是資料：換一份目錄，模塊不變	CL / commit 歷史+症狀目錄 → 訊號與事件 沙盒：十六種裡從歷史看得出的都要發事件
依據分級的認知 PROJECT_MAP、人的地圖 草稿、日誌	每個記下來的事都標它怎麼來的：擷取 / 實跑 / 文件 / 告知 / 推測 推測沒經人確認，不能當行動的依據	任何觀察 → 帶依據的紀錄 沙盒：拿「推測」去動手要被擋下
交付物推導 CD 的第一步 (D12)	從引用與歷史推出一個目錄交什麼、給誰、什麼形式、多久一次 成為可被 owner 確認的草稿，不是結論	地圖+label+下游引用 → 交付物草稿 沙盒：下游引用了的檔要出現在草稿裡

看的四個模塊建立 agent 對 depot 的認知，每一筆都帶依據；推測沒經人確認不能當依據。

任何要進一個陌生 depot 的 agent 都要這四個；它們只靠「接 depot 的程式」接公司。

判：agent 用三個模塊照評分表打分，每條附檢驗、重現要能辨認結果、不排名

模塊 用在哪	規格（建立什麼性質；不講機制）	進出 沙盒怎麼驗
照評分表打分 六個檢查、成熟度 基線	給一張「每條有做得到 / 做不到檢驗」的表， 就能對一個 depot 逐條判定，附證據與依據 表是資料：六原則是一張，資安、coding 規範是另外的張；模塊不變	depot + 評分表 → 分數表（做得到 / 部分 / 做不到） 沙盒：六原則表的每條在埋的問題上判成做不到
乾淨環境重現 SSOT、sanity CD 的「拿了就能跑」	主要文件是否描述了怎麼重現執行， 而且結果有 fail-safe 的辨認方式 三種結果：過 / 不過 / 探測本身壞了 失敗分類：缺檔 / 缺工具 / 缺環境 / 缺 script	depot 的說明 + 一個乾淨環境 → 判定 + 失敗分類 沙盒：三種 mock check 各回對的結果；缺檔要分類成缺檔
趨勢量測 基線、四個指標 給 PM 的摘要	同一張表、同一組指標對同一目錄隨時間量 只看趨勢、以目錄為尺度，不排名個人	分數表 + 指標 → 時間序列 沙盒：兩次掃描的差異算得出；輸出不得出現人名排序

判的三個模塊把評分表套在 depot 上：表是資料、模塊不變；重現的結果要能辨認；趨勢不排名。
「乾淨環境重現」一條規格，SSOT 的檢查、sanity check、CD 的「拿了就能跑」都是它。

做：三個模塊讓 agent 把沒過的檢驗寫成提案；任何 check 先只報告，有共識才擋

模塊 用在哪	規格（建立什麼性質；不講機制）	進出 沙盒怎麼驗
缺口→補丁 shelved CL 計畫核准的請示	每個沒過的檢驗對應一個最小、附證據的提案： 為什麼、怎麼驗、影響誰、怎麼退回 分成三類：自己能補 / 要人決定 / 只有人知道	分數表 → 提案清單（含三類分工） 沙盒：每個提案掛對原則、分對類、附證據
證據打包 Traceability、CD 出包 agent 自己的交付	任何結果附一份清單，能回答 哪一版、什麼工具、什麼環境、怎麼跑 由產生結果的步驟順手產出；通過就標 known-good	結果 → manifest、known-good 標記 沙盒：沒 manifest 的交付視為未完成
逐級收緊 上線分級的階段 trigger	任何自動 check 都走只報告→警告→擋；升級要證據（誤報率） 擋要同意+bypass+總開關；吵了就退一級	一道 check → 它現在在哪一級 沙盒：沒 bypass 時嘗試擋，要被拒

做的三個模塊把缺口變成附證據的最小提案、把結果打包成證據、把自動 check 逐級收緊。

三個都不碰 depot 的內容：提案給人收、證據跟著結果走、擋要人同意。

說：三個模塊管 agent 和人的互動：PM 看懂才核准、不濫發訊息、動作留紀錄

模塊	用在哪	規格（建立什麼性質；不講機制）	進出	沙盒怎麼驗
理解後核准	計畫核准、擋 submit (D3)	要人簽的事，先附原則、影響誰、怎麼退回，再用兩個問題確認對方懂了 答不到不算核准；逾時不自動通過；分批送	提案 → 請示 + 核准紀錄	沙盒：假 PM 答不到「影響誰」，agent 要再問
節制的聯絡	問 owner、提醒更正、開工的那一句	先查到能查的才開口；只問對方才知道的；同一件事不帶新資訊不重問 每人每天有上限；先私訊；錯了在原處更正	要說的事 + 對象 → 訊息，或不發	沙盒：同一問題第二次要帶新資訊，否則不發
可查的行動日誌	透明、idempotent	每個動作、判斷、訊息都記看了什麼、推了什麼、不確定什麼 當事人看得到關於自己的；重啟從日誌接續、不重複	動作 → 日誌（有 schema 版本）	沙盒：中途重啟，不重複問、不重複開 CL

說的三個模塊管 agent 和人共處：PM 看懂才核准、不濫發訊息、每個動作留可查的紀錄。

這三個和 CI/CD 無關，任何會跟人打交道的 agent 都該拿去用。

守：四個模塊管住 agent：規矩明寫、範圍先登記、改自己先過沙盒、花費有上限

模塊 用在哪	規格（建立什麼性質；不講機制）	進出 沙盒怎麼驗
目錄的規矩登記 review 規矩 (D7) 交付物 (D12)	每個目錄的規矩要明寫：要不要 review、交付物是什麼、誰負責 沒寫就標「未定」並去要一個，不自己猜	目錄 → 規矩，或「未定」 沙盒：沒寫規矩的目錄要出現「未定」的提問
範圍登記與獨占 多實例、換手	一個範圍任何時候只有一個行動者；共用物指定負責 動手前先查；換手有交接包	範圍 + 行動者 → 登記表、HANDOVER 沙盒：兩個實例同一路徑，第二個要退
自我版控與沙盒 core / 工作區 release、換手	行動者不改自己運行中的規則；改進走 review (MR)、 過埋了已知問題的沙盒、有人看過才出 release 每個 release 三軸驗收：偵測到了嗎、提案對嗎、沒做不可做的事	改進 → MR → 沙盒 → release 沙盒：就是沙盒本身
預算與總開關 kill switch 多實例的代價	每個行動者有訊息、算力、token 的預算，超過先停、告知 它裝的機制有一個人按得到的總開關	預算 + 開關 → 停或繼續 沙盒：超預算要停並告知；開關按下去機制全停

守的四個模塊讓 agent 管住自己：規矩要明寫、範圍要登記；改自己走 review 與沙盒；有預算、有總開關。

這四個讓多個 agent 能同時存在而不互相踩、不失控。

接公司：公司專屬的只有接 depot 的程式、評分表、規矩表；換掉就是別的 agent

模塊	靠公司專屬三樣的哪一樣接公司（-= 完全通用）	別的 agent 怎麼用
結構地圖	接 depot 的程式	任何 depot 的入門地圖
變更流觀察	接 depot 的程式、症狀目錄（規矩表）	換一份症狀目錄：資安事件、命名違規
依據分級的認知	—	任何會記東西的 agent
交付物推導	接 depot 的程式	任何有上下游的目錄
照評分表打分	評分表	換表：資安、coding 規範、文件完整度
乾淨環境重現	接 depot 的程式（要一個乾淨環境）	任何「說明能不能重現」的檢查
趨勢量測	評分表、指標	任何要看改善的推動
缺口→補丁	接 depot 的程式、評分表	任何「不過就提案」的 agent
證據打包	接 depot 的程式	任何產出結果的 flow
逐級收緊	規矩表	任何自動 check 的推行
理解後核准	規矩表（授權）	任何要人簽的 agent
節制的聯絡	規矩表（上限）	任何會私訊人的 agent
可查的行動日誌	—	任何自主運行的 agent
目錄的規矩登記	規矩表	任何有「每個目錄各自定」的規則
範圍登記與獨占	規矩表（登記）	任何多實例的 agent
自我版控與沙盒	—	任何會改版的 agent
預算與總開關	規矩表（預算）	任何會花錢、裝機制的 agent

十七個模塊接公司的只有三樣：接 depot 的程式、評分表、規矩表；三個模塊完全通用。

換評分表、換規矩表，就是另一個 agent；目標換成 git，也只換「接 depot 的程式」這一樣。

組裝：mentor agent 的十個階段各挑幾個模塊，所以模塊能分開做、分開驗

mentor agent 的階段	用到的模塊（階段的順序照《互動場景》，從啟動到換手）
啟動	可查的行動日誌、節制的聯絡（自我介紹）、範圍登記與獨占、預算與總開關
盤點	結構地圖、變更流觀察、依據分級的認知、照評分表打分、趨勢量測（基線）
目標分析	結構地圖、乾淨環境重現、照評分表打分、交付物推導、目錄的規矩登記、缺口→補丁
計畫核准	理解後核准、節制的聯絡、可查的行動日誌（核准紀錄）
建置	缺口→補丁、證據打包、乾淨環境重現、節制的聯絡（交 shelved CL）
上線分級	逐級收緊、證據打包、趨勢量測、預算與總開關（擋之前先有總開關）
交付（CD）	交付物推導、目錄的規矩登記、證據打包、缺口→補丁
監看與開工	變更流觀察、依據分級的認知、節制的聯絡、目錄的規矩登記、可查的行動日誌
擴充與交棒	趨勢量測、逐級收緊、理解後核准、自我版控與沙盒（新實例從 core clone）
換手與改版	自我版控與沙盒、範圍登記與獨占、可查的行動日誌

mentor agent 的十個階段各用三到六個模塊；沒有一個階段用到全部，每個模塊至少被兩個階段用。

所以模塊能分開做、分開在沙盒驗、分開換版；步驟按順序排，模塊可以在任何階段被叫。